ОПРЕДЕЛЕНИЯ, ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

1. ИИ (искусственный интеллект) – это механизм, работающий на основе автоматического и программного управления, алгоритмы которого задаются заранее, а не генерируются внутренне самой системой, руководствуясь описаниями, заданными в виде шагов и действий, основываясь на информации из внешней среды; [3]

2. БС – Байесовская сеть

ВВЕДЕНИЕ

В эпоху цифровой трансформации данные стали ключевым ресурсом для анализа, машинного обучения и принятия решений. Однако использование реальных данных, особенно в чувствительных областях (медицина, финансы, персональная аналитика), сталкивается с проблемами конфиденциальности и законодательных ограничений, таких как GDPR и ФЗ-152 «О персональных данных».

Цель выпускной квалификационной работы — разработка метода обеспечения дифференциальной приватности при генерации синтетических данных.

Для достижения поставленной цели необходимо решить следующие задачи:

— проанализировать и классифицировать существующие методы генерации синтетических данных и дифференциальной приватности;

— сформулировать концепцию комбинированного метода на уровне идеи, формализовать задачу в виде функциональной модели;

— выделить ключевые этапы разрабатываемого метода и описать их в виде детализированной диаграммы IDEF0 и схем алгоритмов;

— разработать программное обеспечение, реализующее разработанный метод;

— исследовать характеристики разработанного метода при различных входных данных;

— провести сравнение результатов работы реализованного метода с альтернативными методами.

1 Аналитическая часть

1.1 Синтетические даннные

Синтетические данные – это искусственно сгенерированные компьютером данные, обладающие теми же характеристиками и статистическими свойствами, что и реальные данные. Они применяются в том случае, если получение реальных данных затруднено, требует больших затрат, ставит под угрозу конфиденциальность пользователей или занимает слишком много времени. Синтетические данные могут быть текстовыми или табличными, а также представлять собой видео, изображения или звуки, как и реальные данные.

Синтетические данные разделяют на три категории:

1. Полностью синтетические данные, которые не имеют с реальными данными почти ничего общего и генерируются на основе некоторых признаков реальных данных. Они сохраняют статистические свойства реальных данных и фактически заменяют их.

2. Частично синтетические данные, которые заменяют только некоторые значения в реальных данных, если они являются конфиденциальными или содержат пропуски.

3. Гибридные синтетические данные, которые генерируются с использованием и реальных, и полностью синтетических данных: для реальных данных выбираются близкие им синтетические, и они объединяются, формируя гибридные данные.

Создание синтетических данных включает в себя различные методы и подходы, которые позволяют генерировать данные, максимально приближенные к реальным по своим характеристикам и структуре.

Одним из основных методов является статистическое моделирование, при котором данные генерируются на основе заранее определённых статистических моделей и распределений. Например, можно использовать нормальное распределение для создания данных, похожих на реальные по среднему значению и стандартному отклонению.

1.1.1 Генеративыне модели

Генеративные модели представляют собой преобразующий подход в системах принятия решений, фиксируя сложные распределения данных и генерируя новые экземпляры данных, которые сохраняют исходные характеристики данных, которые они моделируют. Эта внутренняя способность определяет их роль в структурах принятия решений, где прогнозирование новых результатов на основе прошлых данных имеет решающее значение. https://cyberleninka.ru/article/n/razvitie-generativnyh-modeley-v-logicheskih-sistemah-prinyatiya-resheniy/viewer

1.1.2 Статические и вероятностные методы

1.2 Дифференциальная приватность

Дифференциальная приватность обеспечивает защиту личной информации, гарантируя, что результаты анализа не позволят точно определить информацию о конкретных индивидах. В рамках ДП данные обрабатываются с добавлением шума, что минимизирует влияние индивидуальных записей на результаты статистических запросов. Таким образом, как показано на рисунке 1.1, даже если в набор данных добавляется или исключается информация о конкретном индивиде, это не влияет на общие выводы о группе [**Wood2018DifferentialPA**].

Представим БД, содержащую медицинские записи 100 человек, из которых 20 страдают диабетом. Злоумышленник хочет узнать, есть ли у человека X диабет, и уже узнал, что 19 из остальных 99 человек в выборке являются диабетиками. Запрашивая базу данных и получая информацию о том, что в выборке 20 диабетиков, злоумышленник делает вывод, что человек X также страдает диабетом. Этот процесс извлечения дедуктивной информации называется дифференцированной атакой [**harvard**].

Теперь представим, что доступ к данным, хранящихся в БД, предоставляется через систему запросов с дифференциальной приватностью. Когда злоумышленник делает запрос, система возвращает истинное значение с некоторым случайным шумом. Даже если злоумышленник знает все о других 99 людях, то он не может быть уверен, есть ли у человека X диабет. Конфиденциальность человека X защищена, поскольку его риск нарушения конфиден-

циальности остается примерно одинаковым независимо от того, входит ли он в набор данных или нет. Это справедливо для любого отдельного человека и любого набора данных, независимо от того, насколько необычными или отличительными могут быть данные конкретного индивидуума. [**harvard**]

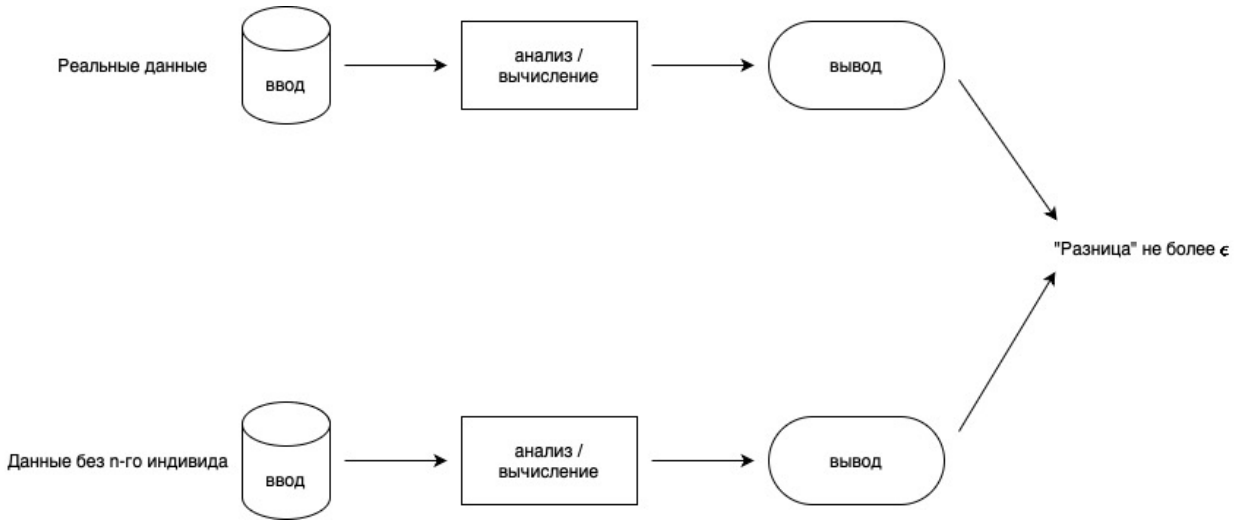


Рисунок 1.1 – Принцип ДП

1.3 Типы приватности

Параметр приватности ϵ — это неотрицательное значение, определяющее уровень приватности, который влияет на количество добавляемого шума [**harvard**].

1.3.1 ϵ – дифференцированная приватность

Модель ϵ – дифференцированной приватности гарантирует, что результаты анализа данных не изменятся, если одна запись будет добавлена или удалена. [**main**]

Механизм M обеспечивает ϵ – дифференциальную приватность, если для любых двух соседних наборов данных D и D' (различающихся на одну запись) выполняется условие по формуле 1.1:

$$Pr[M(D)] \leqslant \exp(\epsilon) \times Pr[M(D')] \tag{1.1}$$

, где ϵ — параметр приватности

1.3.2 (ϵ, δ) – дифференцированная приватность

Модель (ϵ, δ) – дифференцированной приватности вводит дополнительный параметр δ, который ослабляет строгие гарантии приватности. Параметр δ определяет вероятность того, что гарантия приватности будет нарушена в некоторых случаях. Это дает гибкость в балансе между приватностью и

точностью результатов. [**main**]

Механизм M обеспечивает (ϵ, δ) – дифференциальную приватность, если для любых двух соседних наборов данных D и D' (различающихся на одну запись) выполняется условие по формуле 1.2

$$Pr[M(D)] \leqslant \exp(\epsilon) \times Pr[M(D')] + \delta \tag{1.2}$$

, где ϵ — параметр приватности, δ — параметр вероятности нарушения приватности

1.4 Типы механизмов

Параметр чувствительности — это максимальное изменение результата механизма при изменении одного элемента в базе данных.

Чувствительностью механизма M является максимум разности значений механизма для любых двух соседних наборов данных D и D' (различающихся на одну запись) и вычисляется по формуле 1.3:

$$\Delta M = \max(|M(D) - M(D')|) \tag{1.3}$$

, где M — параметр чувствительности.

1.4.1 Механизм Лапласа

Лапласов механизм добавляет к результату запроса шум, сгенерированный из распределения Лапласа [**main**]. Результат Лапласова механизма вычисляется по формуле 1.4:

$$M'(D) = M(D) + \mathrm{Lap}(0, \frac{\Delta M}{\epsilon}) \tag{1.4}$$

где ΔM — параметр чувствительности, ϵ — параметр приватности

1.4.2 Механизм Гаусса

Гауссов механизм похож на Лапласов, но шум добавляется на основе нормального (Гауссова) распределения. К результатам запроса добавляется шум по формуле 1.5:

$$M'(D) = M(D) + \mathcal{N}(0, \sigma^2) \tag{1.5}$$

, где $\mathcal{N}(0, \sigma^2)$ — нормальное распределение.

Параметр σ вычисляется по формуле 1.6.

$$\sigma = \frac{\Delta M \sqrt{2 \ln(1.25/\delta)}}{\epsilon} \tag{1.6}$$

1.4.3 Экспоненциальный механизм

Экспоненциальный механизм работает путем добавления случайности в процесс выбора элементов, где вероятность выбора каждого элемента зависит от его "качества"(оценки полезности) и параметра приватности.

Для двух соседних наборов данных D и D' экспоненциальный механизм M обеспечивает экспоненциальную приватность, если вероятность выбора результата r зависит от его полезности $u(r, D)$, и вычисляется по формуле 1.7

$$P[M(D)] \propto \exp(\epsilon u(r, D)) \tag{1.7}$$

, где ϵ — параметр приватности, $u(r, D)$ — оценка полезности

2 Конструкторская часть

2.1 Предварительная обработка данных

Тип данных является важным компонентом домена атрибутов. Метод поддерживает четыре типа данных и позволяет пользователям явно указывать их. Если тип данных атрибута не указан пользователем, то метод делает это автоматически.

Тип данных атрибута ограничивает его домен, который может быть дополнительно ограничен, если атрибут категоричен. Например, в наборе данных студентов атрибут может принимать только значения BS, BA, MS или PhD. Домен атрибута — $\{BS, BA, MS, PhD\}$. Атрибут считается категориальным, если во входном наборе данных наблюдается ограниченное количество различных значений для этого атрибута.

Этап предварительной обработки данных продемонстрирован на рисунке 2.1



Рисунок 2.1 – Этап предварительной обработки данных

2.2 Построение структуры сети с ДП

На листинге **??** представлен алгоритм, который строит БС $\mathcal{N}$ из входного набора данных D, атрибутов A и максимального количества родителей узла БС k. В этом алгоритме V — это набор посещаемых атрибутов, а Π - это подмножество V, которое станет родителями узла X, если будет добавлено к $\mathcal{N}$. Атрибуты Π определяются путем поиска максимального $I(X, \Pi)$, который вычисляется по формуле 2.1:

$$I(X_i; \Pi) = \sum_{x_i, \pi} P(x_i, \pi) \log \frac{P(x_i, \pi)}{P(x_i)P(\pi)} + Lap(\frac{1}{n\epsilon}) \tag{2.1}$$

, где ϵ — параметр приватности, n — количество записей входных данных.

Algorithm 1: Preprocess

Input : D : датасет;
A : набор атрибутов;
k : максимальное количество родительских узлов;
Output : N : структура Байесовской сети.

begin
- $\mathcal{N} \leftarrow \emptyset$ и $V \leftarrow \emptyset$
- $X_1 \leftarrow {}_R A$
- $A \cup \{(X_1, \emptyset)\}$ и $\mathcal{N} \cup X_1$
- **for** $i \leftarrow 2$ *to* $|A|$ **do**
  - $\Omega \leftarrow \emptyset$
  - $p \leftarrow \min(k, |V|)$
  - **for** *each* $X \in A$ *and each* $\Pi \in \binom{V}{p}$ **do**
    - $\Omega \cup \{(X, \Pi)\}$
  - $(X_i, \Pi_i) = \underset{(X,\Pi)\in\Omega}{\operatorname{argmax}} I(X; \Pi)$
  - $\mathcal{N} \cup (X_i, \Pi)$
- **return** $\mathcal{N}$

2.2.1 Генерация синтетических данных

На начальном этапе генерации метод читает построенную БС. Корневые атрибуты, которые не зависят от других переменных, сэмплируются из их безусловных распределений, а дочерние — из условных. Обход сети происходит в топологическом порядке.

Algorithm 2: Preprocess

Input :

n : количество записей для генерации;
S : метаданные датасета;
s : начальное число.

Output :

$keys$: отсортированный массив нормализованных ключей;
$data$: отсортированный по массиву ключей массив указателей;
cdf : массив значений функции распределения.

begin

установить начальное значение = s для псевдо-случайной генерации чисел
Пример корневого атрибута из безусловного распределения сформировать оставшиеся атрибуты из условного распределения
return Сформированный датасет

2.2.2 IDEF0 уровня A1

На рисунке представлена диаграмма IDEF0 уровня A1 разрабатываемого метода. На вход подается: исходный датасет, на основе которого будут генерироваться синтетические данные, и параметры конфигурации, которые включают в себя ϵ–параметр приватности и количество генерируемых записей.

3 Технологическая часть

3.1 Выбор средств программной реализации

Для реализации метода был выбран язык Python 3.12

3.2 Реализация программного обеспечения

3.2.1 Реализация алгоритма построяения Байесовской сети

Листинги кода

Реализация программы для построения БС с дифференциальной приватностью представлена на листинге ??.

Листинг 3.1 – Исходный код программы Hello World

```
def greedy_bayes(dataset: DataFrame, k: int, epsilon: float, seed=0):
    """Construct a Bayesian Network (BN) using greedy algorithm.

    Parameters
    ----------
    dataset : DataFrame
        Input dataset, which only contains categorical attributes.
    k : int
        Maximum degree of the constructed BN. If k=0, k is automatically calculated.
    epsilon : float
        Parameter of differential privacy.
    seed : int or float
        Seed for the randomness in BN generation.
    """
    set_random_seed(seed)
    dataset: DataFrame = dataset.astype(str, copy=False)
    num_tuples, num_attributes = dataset.shape
    if not k:
        k = calculate_k(num_attributes, num_tuples)

    attr_to_is_binary = {attr: dataset[attr].unique().size <= 2 for attr in dataset}
```

```
22
23      print('================ Constructing Bayesian Network (BN)
            ================')
24      root_attribute = random.choice(dataset.columns)
25      V = [root_attribute]
26      rest_attributes = list(dataset.columns)
27      rest_attributes.remove(root_attribute)
28      print(f'Adding ROOT {root_attribute}')
29      N = []
30      while rest_attributes:
31          parents_pair_list = []
32          mutual_info_list = []
33          num_parents = min(len(V), k)
34          tasks = [(child, V, num_parents, split, dataset) for
                child, split in
35                      product(rest_attributes, range(len(V) -
                            num_parents + 1))]
36          with Pool() as pool:
37              res_list = pool.map(worker, tasks)
38
39          for res in res_list:
40              parents_pair_list += res[0]
41              mutual_info_list += res[1]
42
43          if epsilon:
44              sampling_distribution = exponential_mechanism(
45          epsilon,
46          mutual_info_list,
47          parents_pair_list,
48          attr_to_is_binary,
49                  num_tuples,
50          num_attributes
51          )
52              idx =
                    np.random.choice(list(range(len(mutual_info_list))),
                    p=sampling_distribution)
53          else:
54              idx = mutual_info_list.index(max(mutual_info_list))
55
56          N.append(parents_pair_list[idx])
57          adding_attribute = parents_pair_list[idx][0]
```

```
58            V.append(adding_attribute)
59            rest_attributes.remove(adding_attribute)
60            print(f'Adding attribute {adding_attribute}')
61
62        print('========================== BN constructed
              ==========================')
63
64        return N
```